Algorithm Design - Staff Graded Assignment 1

Name: Amman Jakhar

UID: 2025EB01187

This report documents the design strategy, pseudocode, complexity analysis, test cases and implementation notes for the five problems. Each problem is implemented in a separate C source file (Problem1.c .. Problem5.c). All programs compile with: cc -Wall -std=c11 -o prog ProblemN.c

Problem 1: Empirical Algorithm Analysis - Insertion Sort

Approach / Design Strategy

Insertion Sort uses the incremental (decrease-and-conquer) strategy: it grows a sorted prefix one element at a time, inserting each new key into its correct position among the already-sorted elements. It is the natural choice for studying how input order affects comparison count because its work depends directly on how far each key must move.

What is counted as a key comparison

Every evaluation of the condition (arr[j] > key), i.e. each comparison between the current key and an already-sorted array element - including the test that fails and ends the inner loop. This is the dominant operation of the algorithm.

Algorithm / Pseudocode

```
for i = 1 to n-1:
key = A[i]; j = i-1
while j >= 0:
count++ // key comparison A[j] > key
if A[j] > key: A[j+1] = A[j]; j--
else: break
A[j+1] = key
```

Complexity Analysis

- Best case (already sorted): O(n) - exactly n-1 comparisons.
- Worst case (reverse sorted): O(n^2) - n(n-1)/2 comparisons.
- Average case (random): O(n^2).
- Auxiliary space: O(1) - in-place.

Test Cases / Results (n = 10)

```
Already-sorted : 1 2 3 4 5 6 7 8 9 10 -> comparisons = 9
Reverse-sorted : 10 9 8 7 6 5 4 3 2 1 -> comparisons = 45
Random : 14 31 64 78 6 60 88 95 38 95 -> comparisons = 19 (sample run)
```

All three outputs are correctly sorted ascending. The random case uses srand(time(NULL)), so its exact count varies per run (it lies between the best and worst bounds).

Empirical vs Theoretical

Sorted case gave 9 = n-1, matching the O(n) best case exactly. Reverse case gave 45 = n(n-1)/2 = 10*9/2, matching the O(n^2) worst case exactly. The random run (19) falls between these bounds, consistent with the average-case O(n^2) behaviour. The empirical counts therefore agree with theory.

Implementation Notes

A shared size (N=10) is used for all three arrangements. Each test copies from a master array so the comparison counts are measured on identical data sizes.

Problem 2: Divide and Conquer - Maximum Subarray

Approach / Design Strategy

Divide and conquer. The array is split at the midpoint and the maximum-sum contiguous subarray is the best of three possibilities: entirely in the left half, entirely in the right half, or crossing the midpoint. The crossing case is solved in linear time by expanding outward from the midpoint.

Algorithm / Pseudocode

```
maxSubarray(A, low, high):
if low == high: return (A[low], low, high)
mid = (low+high)/2
L = maxSubarray(A, low, mid) // left half
R = maxSubarray(A, mid+1, high) // right half
C = maxCrossing(A, low, mid, high) // crossing mid
return best of L, R, C
```

Complexity Analysis

- Time: T(n) = 2T(n/2) + O(n) = O(n log n) (Master Theorem, case 2).
- Auxiliary space: O(log n) for the recursion stack.

Indexing Convention

Indices are 0-based, and both start and end indices are inclusive: the result is A[start .. end].

Test Case / Result

```
Input : -2 1 -3 4 -1 2 1 -5 4 (0-based)
Maximum sum : 6
Start index : 3 End index : 6
Selected subarray : 4 -1 2 1
```

The maximum-sum subarray is {4, -1, 2, 1} with maximum sum 6, matching the expected result.

Implementation Notes

The result (sum, lo, hi) is returned via a small struct so the exact indices of the chosen subarray are tracked through the recursion. The given test array was not fixed in the prompt, so the standard CLRS array is used; changing the arr[] initializer in main adapts it to any input.

Problem 3: Divide and Conquer Sorting - Merge Sort

Approach / Design Strategy

Merge Sort (chosen over Quick Sort) is a classic divide-and-conquer sort. DIVIDE: split the array at the midpoint. CONQUER: recursively sort each half. COMBINE: merge the two sorted halves into one sorted run.

What is counted

Key comparisons performed during the merge step - each comparison (left[i] <= right[j]) between an element of the left half and an element of the right half.

Algorithm / Pseudocode

```
mergeSort(A, l, r):
if l < r:
m = (l+r)/2
mergeSort(A, l, m); mergeSort(A, m+1, r)
merge(A, l, m, r) // compares left/right, counts comparisons
```

Complexity Analysis

- Best / Average / Worst time: O(n log n) - the split and merge cost is independent of input order.
- Auxiliary space: O(n) for the temporary merge buffers, plus O(log n) recursion stack.

Test Case / Result (11 integers)

```
Before : 38 27 43 3 9 82 10 1 56 24 17
After : 1 3 9 10 17 24 27 38 43 56 82
Key comparisons during merge = 29
```

Divide / Combine Explanation

The input is repeatedly halved until single-element subarrays remain (trivially sorted). Pairs of sorted subarrays are then merged by repeatedly taking the smaller front element, producing progressively larger sorted runs until the whole array is sorted.

Problem 4: Fractional Knapsack - Greedy

Approach / Design Strategy

Greedy by decreasing value/weight ratio. Because items are divisible, filling the knapsack with as much of the highest value-per-unit-weight item as possible, then the next, and so on, is optimal. An exchange argument confirms that any unit of capacity is best used by the remaining item with the largest ratio - so the greedy choice leads to a global optimum (unlike 0/1 knapsack).

Algorithm / Pseudocode

```
compute ratio = value/weight for each item
sort items by ratio (descending)
for each item while capacity remains:
if item fits: take whole item; capacity -= weight
else: take fraction = capacity/weight; capacity = 0
```

Complexity Analysis

- Time: O(n log n) dominated by sorting by ratio; selection pass is O(n).
- Auxiliary space: O(1) beyond the item list.

Test Case / Result (capacity = 50)

```
Item Weight Value Ratio Fraction
1 10 60 6.00 1.00
2 20 100 5.00 1.00
3 30 120 4.00 0.67
Maximum value achievable = 240.00
```

Items 1 and 2 are taken fully (60 + 100 = 160, using 30 of 50 units). The remaining 20 units

take 20/30 = 0.67 of item 3, adding 0.67*120 = 80, for a total of 240 - matching the expected result.

Implementation Notes

Capacity was not fixed in the prompt text, so the standard value 50 is used; it is a single named variable in main and easily changed.

Problem 5: Job Sequencing with Deadlines - Greedy

Approach / Design Strategy

Greedy by decreasing profit with latest-slot placement. Jobs are considered from highest profit downward; each selected job is placed in the latest free time slot on or before its deadline. Scheduling as late as possible keeps earlier slots free for jobs with tighter deadlines. An exchange argument shows this never reduces total profit, so the schedule is optimal. Each job takes one unit of time and no two jobs share a slot.

Algorithm / Pseudocode

```
sort jobs by profit (descending)
for each job:
for t = deadline downto 1:
if slot[t] free: assign job to slot[t]; add profit; break
```

Complexity Analysis

- Time: O(n log n) to sort + O(n * maxDeadline) for slot search = O(n^2) worst case with linear slot scanning.
- Auxiliary space: O(maxDeadline) for the slot table.

Given Jobs

```
J1(d=2,p=100) J2(d=1,p=19) J3(d=2,p=27) J4(d=1,p=25) J5(d=3,p=15)
```

Test Case / Result

```
Slot 1 : J3 (profit 27)
Slot 2 : J1 (profit 100)
Slot 3 : J5 (profit 15)
Final job sequence : J3 J1 J5
Total profit = 142
```

Processing by profit: J1(100) -> slot 2; J3(27) -> slot 1; J4(25) -> deadline 1 taken, rejected; J2(19) -> deadline 1 taken, rejected; J5(15) -> slot 3. Total profit = 100 + 27 + 15 = 142, matching the expected maximum.

Implementation Notes

The slot table is sized to the maximum deadline. Any ordering achieving profit 142 is an equally valid optimal solution.

Assumptions Summary

- Problem 2 indices are 0-based inclusive; standard CLRS test array used.
- Problem 3 implements Merge Sort and counts merge key comparisons.
- Problem 4 uses knapsack capacity 50.
- Problem 1 random case varies per run due to srand(time(NULL)).